



National University

of Computer & Emerging Sciences



Program: BSCS
Semester: Spring 2025
Course: Object Oriented Programming
Instructor : Dr. Bilal Khan

Assignment No. 01
Total Marks: 10
T.A: Abdul Rafy

Note:

- First think about a problem statement and then write/draw your logic on paper.
- After designing the logic on paper, code the problem statement on any editor (VS Code, Gedit, etc).
- Copied tasks will be awarded zero marks without any investigation.
- Comments you code properly.
- Assignment After Due Date will not be Accepted.
- Plagiarism of any shape or form will not be tolerated. In case of plagiarism, the particular question will be marked zero and 50% marks from total obtained marks will be deducted....

Submission Guidelines:

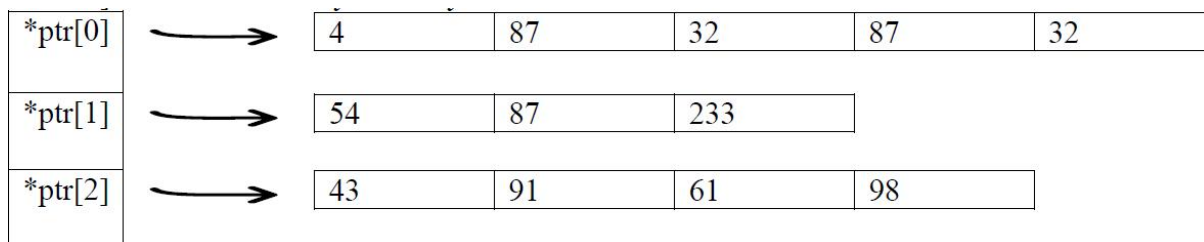
- Submit your .cpp file with proper comments explaining each part of your code.
- Submit only the **<rollno_yourname.cpp>** file, 1 mark will be deducted otherwise.
For example 23p-0560_abdul-Rafy.cpp. 0 marks will be assigned to plagiarized

Task 1

Design a C++ program that define a 2-Dimensional array with different number of values in each row. You need to fulfil all the requirement as given below,

- You must use double pointer to define and access the elements of array
- The system asks how many rows **n** the user want to define
- The system asks how many elements should have for each row [store those numbers in 1D array (DMA)]
- Define that rows with required amount of values in each row (already stored in 1D array name cols)
- Input values for each row one by one in 2D array
- Print all the values row wise from 2D array
- Delete the 2D array
- Delete the pointer array

For example, if user enter n=3 then you should have 3 rows in total, then enter cols = [5, 3 and 4]. The structure of your array should be like



Task 2

Imagine you've been hired by a prestigious university to develop a Student Management System that will help the administration manage student data efficiently. The system should handle a dynamic list of students because new students enroll each semester while others graduate. The university's primary requirement is that the system must handle these operations dynamically without any fixed-size data structures.

Objectives:

- **Data Storage Using Structures:**

1. Define a structure named Student to hold individual student information.
2. The structure must include:
 - id (integer): A unique student identification number.
 - name (character pointer): The student's name, which will be allocated memory dynamically.
 - age (integer): The student's age.
 - gpa (float): The student's Grade Point Average.
 - credits (integer): The total number of credits the student has completed.

- **Dynamic Memory Allocation:**

- Ask the user to input the total number of students n that will be added to the system.
- Dynamically allocate memory for n student records using pointers.
- Since student names can vary in length, allocate memory dynamically for each name instead of using fixed-size arrays.

Functionalities:

1. Add Student Records:

- Input and store data for n students.
- Ensure memory is allocated dynamically for names and the array of student records.

2. Display All Records:

- Print all stored student records in a clear and readable format.
- Include all fields like ID, Name, Age, GPA, and Credits.

3. Find the Top Performer:

- Identify the student with the highest GPA.
- Display their complete information.

4. Sort Records by Credits (Descending Order):

- Sort the student records based on the number of credits completed in descending order.
- Use pointer manipulation instead of indexing or creating temporary arrays.

5. Search for a Student by ID:

- Allow the user to enter a student ID and search for the corresponding student's record.
- Display a message if the student ID is not found.

6. Update Record:

- Provide an option to update a student's GPA by entering their ID.
- Display a message if the student ID is not found.

7. Delete Student Record:

- Allow deletion of a student record based on the ID.
- After deletion, adjust the dynamically allocated memory to ensure no memory leaks.

Constraints & Rules:

- No global variables are allowed.
 - No fixed-size arrays for storing names—use dynamic memory allocation for names.
 - Avoid using indexing where pointer manipulation is possible.
 - Properly deallocate all dynamically allocated memory before the program terminates to prevent memory leaks.
 - The program must handle edge cases, such as:
 1. Attempting to delete a non-existent student.
 2. Updating the GPA of a student that doesn't exist.
 3. Handling cases when there are no students in the system.
-